Designspecifikation - Othello

TSEA83, Grupp 36

Frida Strömberg - Frist757
Ivar Lenner - Ivajo851
Embla Olin - Embol439
Lovisa Bogeklint - Lovbo105

Version 1.0
Mars 11, 2025

Sammanfattning

Denna designspecifikation beskriver konstruktionen av en processor som är anpassad för spelet Othello. Processorn består av en 2-stegs pipeline och ett uppdelat program och dataminne, vilket möjliggör parallell åtkomst till både instruktioner och data. Hårdvaran inkluderar specifika enheter som är ansvariga för olika uppgifter, bland annat som att hantera beräkningar och minnesoperationer. Varje enhet i processorn har specifika ansvarsområden, och användningen av ett uppdelat minne för program påverkar designen. Processorn arbetar med 16-bitars ord och programmet lagras i ROM som kommer kunna exekveras direkt vid start. Inmatningen sker via ett tangentbord, och grafiken hanteras via en VGA-skärm som uppdateras vid varje spelardrag. Vissa utmaningar, såsom pipline hantering och eventuell avsaknad av dataforwarding kommer kräva särskild uppmärksamhet i konstruktionen.

Instruktionsuppsättning

Följande instruktioner kommer troligtvis användas för att effektivt kunna hantera spelets logik och uppdatera spelplanen med extra vikt vid snabba minnesåtkomster:

Load	Läser in data från given minnesadress och lagrar det i ett register
Store	Skriver in innehållet från ett register till angiven minnesadress
Add	Adderar värdet från två register och lagrar resultatet i ett register. Uppdaterar flaggorna för Z,N,C,V
Addi	Adderar ett registervärde med en konstant och lagrar resultatet i ett register. Uppdaterar flaggorna Z,N,C,V
Sub	Subtraherar värdet i ett register från ett annat och lagrar resultatet i ett register. Uppdaterar flaggorna Z,N,C,V
Subi	Subtraherar en konstant från ett register och lagrar resultatet i ett register. Uppdaterar flaggorna Z,N,C,V
Mul	Multiplicerar värdena i två register och lagrar resultatet i ett register. Uppdaterar flaggorna Z, N, C, V.
LSR	Logisk högerförskjutning av värdet i ett register. Flyttar alla bitar i ett register åt höger och fyller på med nollor från vänster
Bra	Ovillkorligt hopp till en given adress
Beq	Villkorligt hopp till en adress om Z=1. Senaste resultatet var noll.
Bne	Villkorligt hopp till en adress om Z-flaggan inte är satt. Dvs Z=0

Cmp	Jämför två register genom att subtrahera dem utan att lagra resultatet. Uppdaterar flaggorna Z,N,C,V beroende på resultatet
Push	Lägger ett registers värde på stacken och uppdaterar stackpekaren.
Pop	Hämtar ett registers värde på stacken och uppdaterar stackpekaren.
Reti	Hoppar tillbaka till instruktionen då vi fick ett avbrott, återställer alla flaggor och möjliggör nya avbrott.

För processorn har vi valt den naturliga ordbredden till 16 bitar, vilket ger en balans mellan enkelhet och prestanda. En 8-bitars ordbredd skulle innebära svårigheter i att adressera större minnesmängder och utföra komplexa beräkningar effektivt. Dessutom skulle en 32-bitars CPU vara onödigt avancerad för vår implementation.

I vår implementation kommer vi stödja fyra olika adresseringsmoder, vilka är: direkt, indirekt, immediate och indexerad adressering. Vi har valt ett instruktionsset med en ordbredd på 32 bitar. Formatet består av t.ex. 7+6+6+6+7 eller 7+6+6+13 beroende på antalet register som används i instruktionen. För att förbättra prestandan planerar vi att hantera inmatning med interrupts. Detta för att möjliggöra snabbare responstid för knappar på tangentbordet.

Programmet ska vara laddat på processorn från start och lagras i ett ROM. Därmed kan processorn börja exekvera direkt utan att behöva ladda in programmet från en extern källa. Detta kan medföra färre felkällor, eftersom koden alltid kommer finnas tillgänglig. Därför riskeras inte programmet att misslyckas vid laddning. Nackdelen med det här är dock att varje ändring i programmet kräver att hela processorn syntetiseras om.

Inmatningen sker via ett tangentbord som är kopplat till processorn via USB. Vi vet att en input har kommit då tangentbordets startsignal ger en extern interrupt, varpå processorn kan läsa in och hantera inmatningen.

CPU:ns huvudsakliga uppgifter är att ta emot tangenttryckningar samt uppdatera spelplanen inom rimlig tid.

Processorn ska ta emot tangenttryckningar och kunna uppdatera en spelplan på 8x8 rutor inom en rimlig tid. Vi räknar med att processorns prestanda kommer räcka för de tänkta uppgifterna.

Grafik:

Grafiken hanteras via en VGA-skärm vars upplösning liknar den från labb 4. Vi använder tre färger: vit, svart och grön. Tanken är att uppdatera skärmen vid varje knapptryck, så att spelaren kan se vilken ruta den är på för tillfället, samt då en bricka har lagts. Efter varje drag ska resterande brickor vändas, antingen en i taget genom en fördröjning eller alla på en gång.

Programmering

För att utföra projektet kommer ALU, PC, register och minnesåtkomst behöva programmeras, samt ett instruktionsset med alla operationer som kan tänkas användas under projektets gång,

instruktionssetet diskuteras mer under rubriken Instruktionsuppsättning. Tanken är dessutom att implementera en Assembler för CPU:n som programmerar spelets logik. Utöver detta kommer programminne, dataminne, videominne, samt tangentbordsimatning, som sker via UART, att implementeras och programmeras i VHDL.

Kollisionshantering

Ytterligare något som ska programmeras för projektet är kollisionshantering, så att spelaren varken ska kunna placera en bricka på en ruta där det redan ligger en bricka, eller utanför spelplanen. Kollisionshanteringen är tänkt att implementeras i mjukvaran och inte bland instruktionssetet.

I/O & minnesanvändning

För att ta emot input till spelet kommer ett tangentbord användas, detta kommer ta emot knapptryckningar som kommer tolkas med VHDL-koden, för att sedan omvandlas till skankoder som sedan skickas vidare för att tolkas och skicka output till VGA-skärmen.

I/O-registren kommer vara en minnescell, vilket innebär att tangentbordsinmatningen kommer lagras i en specifik minnesadress med en ”load immediate”-instruktion. Den specifika minnesadress som processorn sedan kan läsa från kommer antagligen vara ett särskilt register i registerminnet. På grund av detta kommer tangenttryckningarna hanteras genom att läsa från minnet istället för att direkt interagera med CPU-registret. Därför kommer vi lägga till en load

CPU:n kommer ha register för att lagra temporärt minne och hantera beräkningar. Det är tänkt att programkoden ska lagras i programminnet (ROM) och att spelplan, poängräkning och annan logik gällande spelet ska lagras i dataminnet (RAM). Utöver detta kommer projektet även innefatta programräknare, adressregister, indexregister och en stackpekare, samt ett videominne för att spara den data som ska matas ut till VGA. Dessa olika register och minnen kommer möjliggöra sparning av värden som används ofta, hantering av stacken, indexerad adresseringen, samt flaggor för jämförelse och avbrottshantering.

Storleken för de olika minnena är fortfarande obestämd, men nu följer några riktlinjer för deras storlekar. Programminnet måste vara stort nog för att åtminstone få plats med assembly-programmet för Othello. Dataminnet måste vara stort nog för att åtminstone få plats med all information gällande brädet (om en ruta är tom, eller innehåller en svart respektive vit bricka). Dessutom behöver registerminnet vara stort nog för att få plats med alla variabler som kan tänkas behövas under projektets uppbyggnad. Några preliminära storlekar är följande:

- PC – 16 bitar
- ADR – 16 bitar
- SP – 16 bitar
- XR – 8-16 bitar beroende på behovet av indexering

När det kommer till vilka minnen CPU:n läser eller skriver i gäller följande:

- Den kommer läsa instruktioner från programminnet (ROM), som kommer vara read-only, och hämta instruktioner därifrån
- Dataminnet (RAM) är tänkt att användas för att CPU:n ska läsa in och skriva ut planets nuvarande tillstånd, samt spelets temporära variabler och räkneoperationer. Dataminnet kommer alltså vara write-and-read. Dessutom kommer dataminnet ha hand om stacken och returadresser vid funktionsanrop. Det som påverkar stacken är instruktionerna ”push” och ”pop”, stacken växer nerifrån och uppåt.
- I/O-minnet kommer läsa in tangentbordsinmatningar från en specifik minnesadress och spara detta, för att videominnet sedan ska läsa av detta och uppdatera skärmen varje drag.

Det är svårt att veta nu hur mycket plats i minnet allt detta kommer ta. Gissningsvis tror vi att det kommer hamna på ca 1 kB ROM och 1-2 kB RAM då Othello har en spelplan på 8x8 rutor, vilket blir 64 bytes om varje ruta skulle lagra 1 byte. Ytterligare minne kommer krävas för stack, temporära variabler och annan logik. Därför uppskattar vi att ca 3 kB + videominnet borde räcka för hela projektet. Registerminnet kommer ha runt 16 stycken register men kan komma att ändras varefter vi implementerar och märker att fler eller färre behövs.

Timing

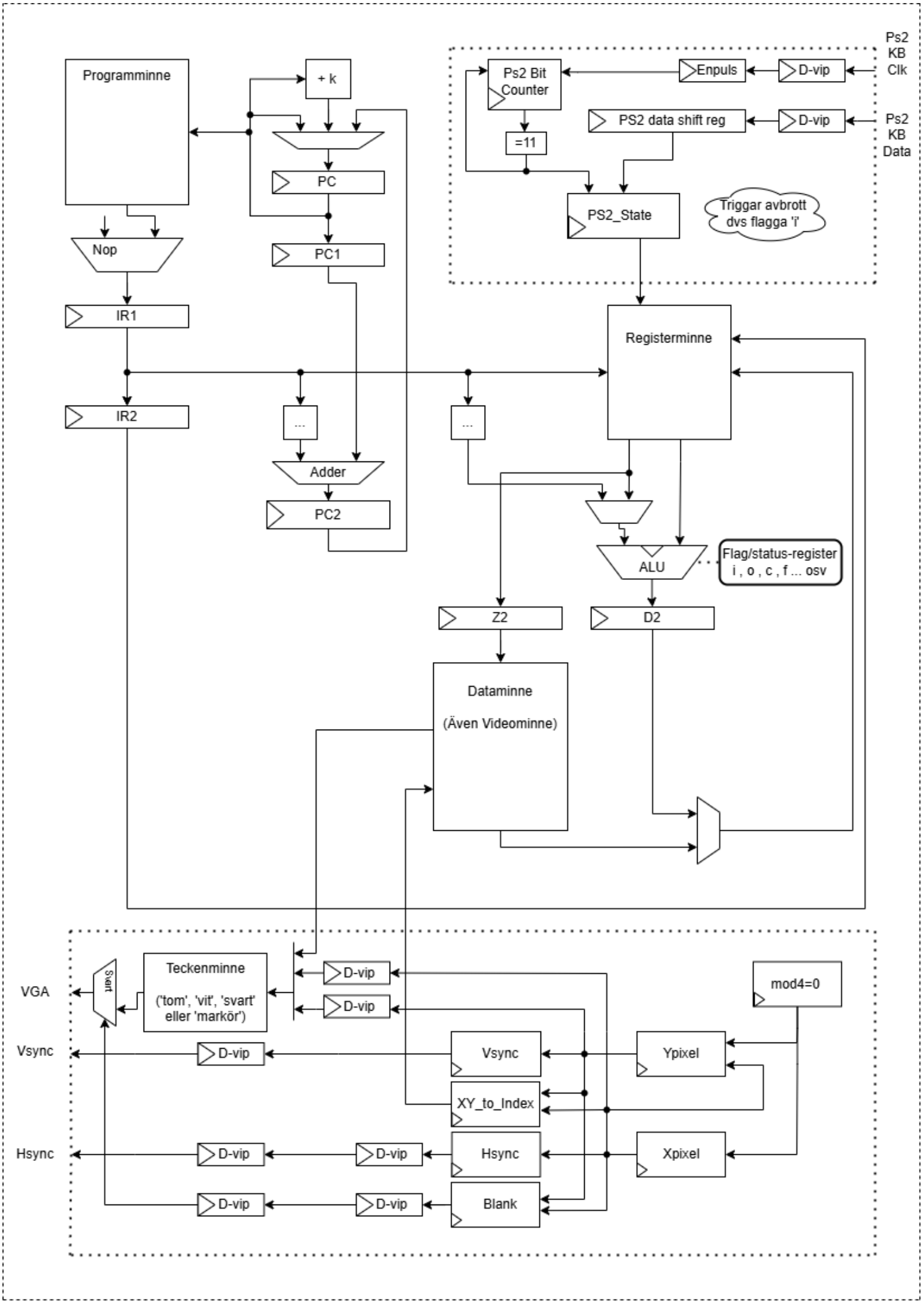
Likt arbetet i labb 4 kommer en räknare implementeras som räknar ett visst antal klockcykler innan den utlöser en viss åtgärd. På så sätt kan processorn fortsätta exekvera instruktioner medan den väntar på att exempelvis flytta ett objekt på skärmen.

Milstolpe

Projektets halvtidsmål är att visa att processorn kan exekvera grundläggande instruktioner, samt att grafikmodulen ska kunna rita ut en spelplan. För att lyckas med detta måste CPU:n kunna hantera ALU-operationer, minnesåtkomst och register- Utöver detta önskas dessutom att tangentbordstryck ska registreras och lagras i minnet

Blockschema

Se nästa blad. Rutorna med titel “…” är menade att hämta ut specifika bitar i en instruktion. Rutan ”Mod 4 = 0” är till för att göra upplösningen mindre men så att datan fyller VGA-skärmen. Siffran 4 är för att en ”storpixel” är 4x4 vanliga pixlar.


Programminne
+ k
PC
PC1
Nop
IR1
IR2
...
Adder
PC2
...
Registerminne
Ps2 Bit Counter
Enpuls
D-vip
Ps2 KB Clk
PS2 data shift reg
D-vip
Ps2 KB Data
=11
PS2_State
Triggar avbrott dvs flagga 'i'
ALU
Flag/status-register i , o , c , f ... osv
Z2
D2
Dataminne
(Även Videominne)
VGA
Svart
Teckenminne
('tom', 'vit', 'svart' eller 'markör')
D-vip
D-vip
mod4=0
Vsync
D-vip
Vsync
Ypixel
XY_to_Index
Hsync
D-vip
D-vip
Hsync
Xpixel
D-vip
D-vip
Blank